ChromSimPipe Code Guide

Cohesin loop-extrusion simulation codebase

2026

ChromSimPipe code guide
Overview
The Snakemake pipeline files
Reading order
Project layout
1. Configuration: `configs/parameters.py`
2. Hi-C data preparation
3. CTCF annotation
4. The simulation model: polysim
5. Running simulations
6. Post-simulation analysis
Legacy cluster scripts (`legacy/cluster/`)
Conventions
Common pitfalls

ChromSimPipe code guide

What each part of the codebase does, what it reads and writes, and how the steps connect. For running the pipeline, see `README.md`.

Overview

What this pipeline does

ChromSimPipe is a physics-based simulation framework for testing mechanistic hypotheses about 3D chromatin organisation. Given CTCF binding data and Hi-C maps, it simulates cohesin loop extrusion under different parameter conditions

and compares the resulting contact maps to experimental Hi-C.

The included example (STRS project, Flores et al. 2026) asks: **can we explain the stress-induced changes in Hi-C contact maps purely from what we can measure (changes in CTCF binding and cohesin dynamics), without invoking anything we can't observe?**

The simulation models cohesin loop extrusion: cohesin rings load onto chromatin and extrude DNA into loops, stalling when they encounter CTCF sites oriented the right way. Where cohesins stall and how long they live determines the 3D contact map. By varying which CTCF sites are present and cohesin dynamics parameters, the simulation tests user-defined hypotheses.

Example conditions (shipped in `config/ChromSimConfig.yaml`):

Condition	Cohesin	CTCF sites	Tests
`cond1_baseline`	Baseline	Condition 1	Does the model reproduce condition 1 Hi-C?
`cond2_ctcf-only`	Baseline	Condition 2	Do condition 2 CTCF sites alone explain condition 2 Hi-C?
`cond2_weak-ctcf`	Baseline	Condition 2 (0.5× stall)	Does weaker stalling at remaining CTCF sites help?

Data flow

```
ChromSimSamplesheet.txt  (one row per sample: metadata,
CTCF_Peaks, Hi-C)
    ↓  orientCTCF rule → scripts/extract_ctcf_sites.py  (strand
via FIMO)
Oriented CTCF BED files (output/loci/<locus>/ctcf/)
    ↓  simulateShard rule → scripts/run_simulation_shard.py +
polysim
Simulated conformations (output/loci/<locus>/sims/)
```

```
    ↓  analyzeCondition rule → scripts/run_analysis_all.py
Comparison figures  ←→  experimental Hi-C (.cool, from
.hic/.mcool/.cool via the convertHic rule)
```

Setup and launch

The workflow follows the PhanstielLab Snakemake pattern (bagPipes, dietJuicer): edit a samplesheet and a config, then submit one launcher. There is no separate setup script: converting Hi-C, orienting CTCF, and building conda environments are all workflow steps.

1. **`ChromSimSamplesheet.txt`**: one tab-separated row per sample. The required columns are `CTCF_Peaks` and `Hi-C` (`NA` when a sample has no file of that kind); all other columns are metadata. Paths are `data/<file>`: each cluster links its own copies into `data/`, so one samplesheet serves every cluster.
2. **`config/ChromSimConfig.yaml`**: `fileNamesFrom` names the samplesheet columns joined with _ to name each sample. Conditions refer to samples by those names (`ctcf_type`, `compare_hic`). The config also sets the locus, physics, genome, CTCF motif, conda environments, and `outputDir`.
3. **`profiles/<site>/`**: the compute site, chosen by `--profile`, then `$CHROMSIM_PROFILE`, then `hpcProfile` in the config. `config.yaml` holds the scheduler, account and each rule's CPUs / memory / GPU under `set-resources:`; `env.sh` loads the software. Shipped: `longleaf` (UNC), `pioneer` (Case Western), `local` (no scheduler), and `slurm-template` (copy and fill in for any other SLURM cluster).
4. **Per-cluster environment variables** (in `~/.bashrc`, outside git): `CHROMSIM_PROFILE` picks the profile and `CHROMSIM_OUTPUT_ROOT` is the folder a relative `outputDir` goes under (e.g. scratch).

```
        sbatch ChromSimPipe.sh        # whole workflow
        bash ChromSimPipe.sh -n       # dry run
        sbatch ChromSimPipe.sh --configfile
config/ChromSimConfig.test.yaml   # 10-job smoke test
```

Test config. `config/ChromSimConfig.test.yaml` is the same config with one condition, one replicate, one shard and 2,000 saved frames, written to `output_test/`. Run it first on any new cluster: it touches every rule, including one GPU simulation, for a fraction of the cost of a full run.

Email. `export CHROMSIM_EMAIL=you@university.edu` and the workflow emails you when it starts, finishes, or fails (see `notify:` in the config).

The central config: `config/ChromSimConfig.yaml`

This YAML file and the samplesheet are the source of truth for everything. `configs/parameters.py` reads both at import time. `loci:` lists one or more windows; every job is told its locus through `$CHROMSIM_LOCUS`, and from that entry the module derives `CHROM`, `REGION_START`, `REGION_END`, and `N_MONOMERS` (1,960 for the default chr1 window at 1 kb per monomer).

CTCF auto-loading. When any script imports `configs.parameters`, it opens the oriented CTCF BED for each CTCF sample and locus if it already exists (e.g. `output/loci/<locus>/ctcf/ctcf_oriented_hg38_condition1.bed`). It parses each line into `(monomer_position, orientation)` where `+1` means a forward-strand motif and `−1` means reverse. If a BED does not exist yet, nothing is loaded, and `get_ctcf_arrays()` raises an error when a script asks for those sites. There are no placeholder sites.

Cohesin parameters. Each condition has a parameter set, a dict with `lifetime` (mean LEF timesteps before cohesin falls off), `separation` (spacing between cohesins) and `ctcf_capture` (probability of stalling per CTCF encounter), optionally overriding `vlef` and `life_boost_stalled` from the `polysim:` block. Defaults are the polysim bootcamp tutorial values.

Tiling. Rather than simulating one copy of the locus, the simulation runs a ~70,000-monomer chain of back-to-back copies (for chr1: 28 copies of 1,960 monomers + 270 padding on each side). The copies are independent, so each GPU run produces ~28× the contact statistics at no extra physics cost. Contact maps are averaged back down to one locus-sized map for comparison. The copy size and count are computed per locus from `tiling.padding` and `tiling.chain_size` (`locus_tiling()`).

The CTCF extraction: `scripts/extract_ctcf_sites.py`

Your `.narrowPeak` file has peak coordinates but no strand information. Orientation matters because a + CTCF site blocks a cohesin arm moving leftward, while a − site blocks one moving rightward. A convergent pair (+ on the left, − on the right) creates a stably anchored loop, while a divergent pair does not.

This script:

1. Takes each peak's summit from the narrowPeak file (column 10).
2. Extracts ± `ctcf_motif.flank_bp` around each summit with `bedtools getfasta`.
3. Runs FIMO for the motif in the MEME file from the `getMotif` rule (JASPAR `ctcf_motif.jaspar_id`, default MA0139.1).
4. Uses the strand of the best hit as the site orientation.

5. Handles peaks with no hit as `ctcf_motif.unoriented` says: `drop` leaves them out and lists them in the log; `error` stops the run. Orientations are never guessed.
6. Writes a BED6 file where column 6 is + or −.

The simulation: `scripts/run_simulation_shard.py` + polysim

The 3D model is the Hansen lab's polysim package (github.com/ahansenlab/polysim_bootcamp), pinned in `envs/cohesin_sim.yml`:

- **Cohesin (1D, Cython).** Each cohesin is a pair of legs that step apart along the chromatin fibre, each with probability `vlef` per LEF timestep. A leg stalls at a CTCF site with the blocking orientation with probability `ctcf_capture` per encounter; legs also block each other. Each cohesin unbinds with probability 1 / `lifetime` per timestep, `life_boost_stalled` times less often while both legs are stalled, and rebinds elsewhere.
- **Polymer (3D, polychrom / OpenMM on GPU).** Each block is `poly_steps_per_block` Langevin steps; the cohesin bonds are springs between the two leg positions, switched on and off as cohesin moves. Every `saveevery`-th block is saved to `blocks_*.h5`. Contacts (pairs within `contact_radius` = 3.0) are counted later, in the analysis.

ChromSimPipe's part (`run_simulation_shard.py`) maps the oriented CTCF sites onto polysim's stall arrays, seeds numpy and C `drand48` per shard, and calls `polysim.sim3d.run()`; see section 5a.

The analysis: `analysis/`

- **`contact_maps.py`**: accumulates saved conformations into a 2D contact frequency matrix using `scipy.spatial.cKDTree` for fast distance lookups. This is the simulated analogue of a Hi-C contact map.
- **`ps_curve.py`**: computes P(s): contact probability as a function of genomic separation. The shape of this curve is the primary comparison metric against real Hi-C.
- **`experimental_compare.py`**: loads your `.cool` file via `cooler` and extracts the same locus at the configured resolution.
- **`msd_two_point.py`**: tracks mean-squared displacement of specific monomer pairs over time (the in-silico analogue of live-cell imaging).

The Snakemake pipeline files

The workflow is a Snakemake DAG written in the style of the PhanstielLab pipelines: a samplesheet plus a config drive everything, every rule writes its own log and benchmark, rules carry no cluster resources (the profile does), and one `sbatch` launches it all.

`ChromSimPipe.sh`: sbatch entry point

Submit with `sbatch ChromSimPipe.sh`. It:

1. Picks the profile (`--profile`, else `$CHROMSIM_PROFILE`, else `hpcProfile`) and sources its `env.sh`, which makes `python3` and `conda` available on that site.
2. Creates a venv in `env/` (first run only) and installs `config/requirements.txt` (Snakemake 8.27.1 + the SLURM executor plugin).
3. Runs `snakemake -s workflows/ChromSimPipe.snakefile --profile <hpcProfile> --configfile <config> --rerun-incomplete --keep-going -p`, adding `--slurm-logdir <outputDir>/logs_slurm` when the profile uses the SLURM executor (`<outputDir>` resolved the same way as in `configs/parameters.py`, including `$CHROMSIM_OUTPUT_ROOT`).

Any extra arguments go to `snakemake` (`-n` for a dry run, `--configfile other.yaml`, `--until validateCTCF`, …).

`sweep_1d.sh`: a cheap 1D screen

`bash sweep_1d.sh` runs `scripts/sweep_1d.py` with the simulation environment (found as in `check_inputs.sh`). For one locus and condition it builds polysim's 1D cohesin engine (`extrusion.build_lef_arrays` + `make_translocator`, with the condition's CTCF stall sites and loading / pausing profiles), runs it for every combination of the given lifetime, separation, ctcf_capture, vlef and life_boost_stalled values, equilibrates for `--lifetimes-equil` (boosted) lifetimes, then records `--frames` snapshots. `score_frame()` scores each snapshot: per loop, the fraction of locus copies where one cohesin has its left leg at one anchor and its right leg at the other (within `--tol` monomers); the mean loop size; and the fraction of the chain inside a loop. Loops are every convergent CTCF pair between `--min-kb` and `--max-kb`, or those in `--loops`. Writes one TSV row per setting. No polymer, no GPU.

`check_inputs.sh`: check the inputs before a run

`bash check_inputs.sh` finds the simulation conda environment (the copy of `envs/cohesin_sim.yml` Snakemake keeps under `.snakemake/conda/`) and runs `scripts/check_inputs.py` with it. That script imports `configs/parameters.py` (so the config and samplesheet are read exactly as in a run) and, for every locus and sample, checks: the locus chromosome is in the genome and the window fits (lengths from `<genome>.fai`, else by reading the FASTA); the CTCF peaks use the same chromosome names, have peaks in each window and none past a chromosome's end; the Hi-C has `resolution`, the same chromosome names, and chromosome lengths identical to the genome's (`.hic` headers are read with hic2cool's `read_header`, `.mcool`/`.cool` with cooler), and for `.mcool`/`.cool` the normalization `analysis.hic_balance` resolves to. It prints PASS / WARN / FAIL lines and exits 1 on any FAIL. Nothing is written.

`unlock.sh`: release lock after failure

Snakemake writes a `.snakemake/locks/` directory while running. If the job dies without cleanup, the lock persists and the next run refuses to start. `bash unlock.sh` runs `snakemake --unlock` against the workflow to clear it.

`workflows/ChromSimPipe.snakefile`: the DAG

Rule	Runs	What it does
`convertHic`	per Hi-C sample	`scripts/prepare_hic.py`: `.hic` (hic2cool), `.mcool` or `.cool` → one `.cool` at `resolution`
`getMotif`	once	`wget` the JASPAR motif (skipped when `ctcf_motif.meme` is set)
`orientCTCF`	per CTCF sample used by a condition	`extract_ctcf_sites.py`
`validateCTCF`	once	`validate_ctcf.py`: sanity-checks all BED files
`simulateShard`	condition × replicate × shard (GPU)	`run_simulation_shard.py`: 18 jobs with the example config
`mergeShards`	condition × replicate	`merge_shards.py`: concatenates shard HDF5 streams
`analyzeCondition`	per condition	`run_analysis_all.py`: contact maps, P(s), APA, figures
`benchmarkSummary`	once	`workflows/utils/benchmarking.py`

Every rule has the same parts, following bagPipes:

- `log:`: stdout / stderr to `output/logs/{rule}_{wildcards}.out|.err`
- `benchmark:`: `output/benchmarks/{rule}_{wildcards}.tsv` (wall time, memory), summarised in `output/{runName}_benchmarking.tsv`
- `conda:`: an entry from `condaEnvs` in the config; the shell runs scripts with `"$CONDA_PREFIX/bin/python"`, that environment's own interpreter
- `params:`: every value read from the config; nothing is typed in

Wildcards: `{sampleName}` / `{ctcfType}` are samplesheet sample names; `{condition}` is a condition `name`; `{rep}` and `{shard}` expand over `range(n_replicates)` and `range(n_shards)`.

Notifications. `onstart:`, `onsuccess:` and `onerror:` handlers run once in the main Snakemake process and call `notify()`, which runs the `notify: command:` from the config with the subject and address filled in and the message on stdin. The address is `$CHROMSIM_EMAIL`, else `notify: email:`; with neither set, nothing is sent. The error message includes the last 40 lines of the Snakemake log. Sending is wrapped so a mail failure can never fail the run, and handlers do not run on dry runs.

The Snakefile passes the config's path to every script in `$CHROMSIM_CONFIG`, so scripts read the same file Snakemake was given. Steps whose real outputs are directories (simulation, merge, analysis) are tracked through completion markers in `output/done/`.

`profiles/<site>/config.yaml`: scheduler settings

This is the only place cluster details live. In `profiles/longleaf/`, `default-resources:` sets the account, partition, wall time, and memory for every job. `set-resources:` sets each rule's request (the dietJuicer pattern). `simulateShard` is sent to whichever of `l40-gpu`, `volta-gpu` or `gpu` has a free node first, with `qos: gpu_access` and `gpu: 1`. `software-deployment-method: conda` makes Snakemake activate each rule's environment.

`ChromSimSamplesheet.txt`: samples

```
Project  Cell_Type  Condition   CTCF_Peaks                Hi-C
STRS     HEK293T    condition1  data/control.narrowPeak
data/control.hic
STRS     HEK293T    condition2  data/sorbitol.narrowPeak
data/sorbitol.hic
```

Relative paths resolve against the repo root, so `data/...` works on any cluster that has linked its files into `data/` (see `data/README.md`).

With `fileNamesFrom: ['Condition']` the samples are `condition1` and `condition2`. Metadata values shared by every row (`STRS`, `HEK293T`) form the run name, `STRS_HEK293T`, used for run-level files. The workflow writes the parsed samplesheet, with a `sampleName` column, to `<outputDir>/{runName}_ChromSimSamplesheet.txt`.

Reading order

Sections are ordered by pipeline stage:

1. The Snakemake pipeline files: **start here**
2. Project layout: what each folder is for
3. Configuration: the single file that controls every simulation
4. Hi-C data preparation: turning experimental data into comparison matrices
5. CTCF annotation: turning CUT&Tag peaks into oriented barrier lists
6. The simulation model: polysim, the loop-extrusion simulator
7. Running simulations: the shard and merge entry points
8. Post-simulation analysis: contact maps, P(s), APA, MSD, calibration
9. Legacy cluster scripts: old sbatch wrappers (superseded by ChromSimPipe.sh)
10. Conventions: monomers, tiling, output directory names
11. Common pitfalls: what to check when something looks wrong

Project layout

```
ChromSimPipe/
├── ChromSimPipe.sh   sbatch launcher
├── ChromSimSamplesheet.txt  one row per sample (edit before
first run)
├── data/             input files, linked in per cluster (git-
ignored except README.md)
├── config/           ChromSimConfig.yaml + requirements.txt
(edit before first run)
├── configs/          parameters.py (reads the config +
samplesheet; single source of truth)
├── envs/             conda environment specs used by the rules
├── scripts/          Python simulation, analysis, and utility
scripts
├── analysis/         Python modules imported by
scripts/run_analysis_all.py
├── workflows/        Snakemake DAG (ChromSimPipe.snakefile) +
utils/
├── profiles/         one folder per compute site (longleaf,
```

```
pioneer, local, slurm-template)
├── legacy/           unmaintained code from the lab's
mESC/neuron project (see legacy/README.md)
├── tests/            test suite
└── output/           everything the workflow writes (git-
ignored; set by outputDir,
                      or under $CHROMSIM_OUTPUT_ROOT when set)
```

1. Configuration: `configs/parameters.py`

Purpose. This file is the runtime source of truth for every simulation parameter. It reads the config (`$CHROMSIM_CONFIG`, default `config/ChromSimConfig.yaml`) and the samplesheet it names at import time, and re-exports a fixed set of variables so that all downstream scripts stay unchanged when config values change.

Key exported names:

- **Loci** (`LOCI`, `LOCUS_NAMES`, `ACTIVE_LOCUS`, `CHROM`, `REGION_START`, `REGION_END`, `RESOLUTION`, `N_MONOMERS`). `LOCI` holds every entry of the `loci:` list; the other names describe the active locus (`$CHROMSIM_LOCUS`, else the first). For the included STRS example: hg38, chr1:64,210,000–66,170,000 → 1,960 monomers at 1 kb/monomer. Each locus is checked on import: unique names without `/`, whole-number coordinates with $0 \le$ start $<$ end, both ends on the `resolution` grid, and, once `<genome>.fai` exists, a chromosome the genome has and an end within it.

- **Samples and outputs** (`SAMPLES`, `RUN_NAME`, `OUTPUT_ROOT`, `OUTPUT_DIR`, `PATHS`, `GENOME`). Sample names, metadata and input files from the samplesheet; every output directory under `outputDir`. `locus_paths(name)` gives any locus' folders (`<outputDir>/loci/<name>/...`; `.cool` files and motifs are shared), `PATHS` those of the active locus. Input paths (samplesheet files, `genome`) resolve against the repo root; a relative `outputDir` resolves against `OUTPUT_ROOT`, which is `$CHROMSIM_OUTPUT_ROOT` if set, else the repo root.

- **Simulation model** (`POLYSIM`). The `polysim:` block: polymer (density, stiffness, repulsion), cohesin bond, extrusion speed (`vlef`), stalled-lifetime boost, integration and saving schedule.

- **CTCF tracks** (`CTCF_TYPES`, `CTCF_PEAKS`, `CTCF_BEDS`). One oriented BED per `ctcf_type` used by a condition, under `output/loci/<locus>/ctcf/`. At import time, `load_ctcf_from_bed()` reads each file into `CTCF_SITES_BY_TYPE[type]`: monomer indices with `+1` / `−1` orientation labels. Type names are samplesheet sample names, so any number of CTCF samples is supported.

- **Cohesin parameter sets** (`ALL_PARAM_SETS`). One dict per condition hypothesis, each with `lifetime`, `separation`, `ctcf_capture`, and optional `vlef` / `life_boost_stalled` overrides.

- **Condition list** (`SIMULATION_CONDITIONS`). The rows the Snakemake pipeline iterates over. Each entry pairs a cohesin parameter set with a CTCF track and has a `name` (e.g. `cond1_baseline`).

- **Tiling** (`TILING`, `locus_tiling(name)`). The locus is copied into one long chain (28× for chr1) for better contact statistics at no extra physics cost. `TILING["chrom_size"]` is the full chain length (~70,000 monomers).

- **Run parameters** (`SIM_RUN`). `seed_base`, `frames_per_replicate`, `contact_radius`, `hdf5_frames_per_file`.

- **Analysis** (`ANALYSIS`, `MSD_PROBE`, `MSD_FIT`, `CALIBRATION`). Worker count, APA window, MSD probes and fits, and how to map simulation units → physical units. All come from the `analysis:` block.

Helper functions:

- `get_condition(name)`: look up one entry in `SIMULATION_CONDITIONS`.
- `list_conditions()`: list all condition names.
- `get_ctcf_arrays(ctcf_type)`: returns `(positions, orientations)` for the untiled locus; raises if that sample's BED has not been made.
- `get_tiled_ctcf_arrays(ctcf_type)`: same but repeated across the tiled polymer (what the simulations actually use).
- `load_ctcf_from_bed(path)`: read a BED6 file and return `(positions, orientations)` as monomer-index arrays.

2. Hi-C data preparation

In ChromSimPipe, Hi-C conversion is handled automatically by the `convertHic` Snakemake rule, which runs `scripts/prepare_hic.py` on each file in the samplesheet's `Hi-C` column to produce a single-resolution `output/mcool/<sample>.cool` at the configured resolution (default 1 kb):

- `.hic`: `hic2cool convert` (Juicer's normalizations become `KR`, `VC`, `VC_SQRT` columns);
- `.mcool`: the `/resolutions/<resolution>` cooler is copied out; the run stops with the resolutions the file has if `resolution` is not among them;
- `.cool`: copied after checking its bin size equals `resolution`.

It prints the normalization columns found. `analysis.hic_balance: auto` uses `KR` if present, else cooler's `weight`.

The Bonev 2017 mESC/cortical-neuron download scripts and the Sox2 quick-look plotter from the original lab project now live in `legacy/` (see `legacy/README.md`); they are not part of the workflow.

3. CTCF annotation

`scripts/extract_ctcf_sites.py`

Purpose. Given a CTCF ChIP-seq or CUT&Tag peak file, a genome FASTA, and a CTCF motif (MEME format), run FIMO to assign strand orientation to each peak in the locus, and write an oriented BED6 file.

The `orientCTCF` Snakemake rule calls this for every sample a condition uses as its `ctcf_type`; every argument comes from the config. To run manually (inside the `ctcf` environment):

```
        python scripts/extract_ctcf_sites.py \
            --bed /path/to/peaks.narrowPeak \
            --region chr1:64210000-66170000 \
            --genome /path/to/genome.fa \
            --motif output/motifs/MA0139.1.meme \
            --assembly hg38 \
            --fimo-thresh 1e-4 --flank 250 --unoriented drop --
resolution 1000 \
            --output
output/loci/chr1_fig1/ctcf/ctcf_oriented_hg38_condition1.bed
```

Output: BED6, one row per peak with a FIMO hit; column 6 (+/−) is the motif strand.

`scripts/validate_ctcf.py`

Purpose. Pre-flight sanity check before any simulation. Loads the oriented CTCF BEDs, converts genomic coordinates to monomer indices, and prints a report. Called by the `validateCTCF` Snakemake rule; also runnable standalone (inside the `sim` environment):

```
        CHROMSIM_CONFIG=config/ChromSimConfig.yaml python
scripts/validate_ctcf.py
```

Checks: BED parses cleanly; all sites are inside the simulation window; coordinate → monomer-index mapping is monotone; prints counts per condition and sites gained / lost / shared. Exits non-zero on failure, including when a BED is missing.

`scripts/export_conserved_ctcf_bed.py`

Purpose. Intersect two oriented CTCF BEDs, keep only sites shared in both (same position and orientation), and write a conserved-sites BED. Useful for testing "CTCF binding didn't change, only cohesin did" scenarios.

4. The simulation model: polysim

ChromSimPipe simulates with the Hansen lab's polysim package (github.com/ahansenlab/polysim_bootcamp, MIT license), pinned to one commit in `envs/cohesin_sim.yml`. It couples a 1D model of cohesin loop extrusion (`LEF_Dynamics.pyx`, Cython) to a 3D polymer on polychrom / OpenMM, as in the polychrom loop-extrusion example and Goel et al. 2025.

polysim setting	Comes from	Default
`life`, `sep`	condition `lifetime`, `separation`	75,000 timesteps, 240 monomers
`vlef`, `lifebooststalled`	`polysim:` block, or per-condition overrides	0.0025, 4
`ctcf_left`, `ctcf_right`	oriented CTCF sites (+ / −), each at the condition's `ctcf_capture`	0.5
polymer, integration, saving	`polysim:` block	bootcamp values
`npoly`	the locus' tiled chain (`TILING["chrom_size"]`)	~70,000
`numsave`	`sim_run.frames_per_replicate / n_shards`	5,000

ChromSimPipe's earlier built-in model (`scripts/lef_dynamics.py`, `scripts/run_simulation.py`) was removed in November 2026: on a CTCF-driven control locus it gave the same contact pattern as polysim but about 7× too weak. It remains in the git history.

5. Running simulations

5a. `scripts/run_simulation_shard.py`: the Snakemake entry point

This is what every `simulateShard` SLURM job calls (18 jobs per locus with the example config: 3 conditions × 3 replicates × 2 shards). The locus comes from `$CHROMSIM_LOCUS`, which the workflow sets.

Arguments:

Flag	Meaning
`--condition NAME`	One of the names in `SIMULATION_CONDITIONS`. Sets cohesin params + CTCF track.
`--replicate N`	Integer replicate index (drives the seed).
`--shard-index I`	Which shard of this replicate (0…`n_shards-1`).
`--n-shards K`	Total shards per replicate.
`--gpu I`	CUDA device index.
`--output DIR`	The locus' sims folder (the workflow passes `output/loci/<locus>/sims`).

Shard output goes to `<output>/<params_name>_ctcf-<ctcf_type>_rep<r>_shard<i>/`.

What it does:

1. Looks up the condition (`get_condition`), works out the shard's frame count (`frames_per_replicate / n_shards`) and seed (`seed_base + replicate × 10000 + shard × 1000`), and writes `params.json`.
2. `seed_all(seed)` seeds numpy and C `drand48()`. polysim's Cython LEF model draws every stall, unbinding and stepping decision from `drand48()` and never seeds it, so without this every shard would replay one random sequence.
3. `ctcf_stall_sites()` turns the tiled CTCF sites into polysim's `ctcf_left` / `ctcf_right` dicts {monomer: stall probability}: + sites stop leftward-moving legs, − sites rightward-moving ones. The probability is `ctcf_capture`, or, for `ctcf_capture: {min, max}`, each site's rank by peak signal (`get_ctcf_signals`, the narrowPeak signalValue of the peak the site came from) mapped onto [min, max].
4. `condition_features()` reads the peak files of the condition's `sticky`, `loading` and `pausing` options (`condition_peak_files`, from samplesheet columns), maps them onto the locus' beads and tiles them

(`scripts/sim_features.py`). Sticky factors become `monomer_types` and an `interaction_matrix` (one type per combination of factors on a bead); loading and pausing become (bead indices, factor) pairs.

5. `polysim_kwargs()` builds polysim's `SimParams`: the tiled chain length, the `polysim:` block, the condition's `lifetime` / `separation` (`None` = no cohesin) and optional `vlef`, `life_boost_stalled`, `attraction_energy`, the stall dicts, and the sticky types. A condition that sets `attraction_energy` without sticky factors gets one bead type, because polysim applies that energy only with its heteropolymer force. It checks that frames per shard × `saveevery` is a multiple of `blocks_per_updater` before any GPU time is spent.
6. `polysim.sim3d.run()`, inside `sim_features.lef_profiles()`, writes `blocks_*.h5` into the shard folder (each saved frame also carries the LEF bonds as `SMCs`, unless there is no cohesin). `lef_profiles` wraps `polysim.extrusion.build_lef_arrays` for that one run, multiplying the loading array (`birth`) at loading peaks and lowering the stepping probability (`1 − pause`) at pausing peaks; polysim itself fills both uniformly.

The `preparePolysim` rule imports polysim once before the shards, so its Cython module (`LEF_Dynamics.pyx`, built by pyximport) is compiled once.

5b. `scripts/merge_shards.py`

Concatenates per-shard HDF5 streams into `merged_<condition>_rep<N>/` with the same layout the analysis expects. Writes `merge_metadata.json` recording which shard IDs contributed.

```
--condition, --replicate, --n-shards, --results-dir
```

6. Post-simulation analysis

6a. The orchestrator: `scripts/run_analysis_all.py`

One command that walks every `merged_<condition>_rep<N>` directory, runs the full analysis suite, and writes per-condition outputs. Called by the `analyzeCondition` Snakemake rule.

How to run directly (e.g. for one condition):

```
        CHROMSIM_CONFIG=config/ChromSimConfig.yaml python
scripts/run_analysis_all.py \
            --results-dir output/loci/chr1_fig1/sims \
            --output-dir
output/loci/chr1_fig1/analysis/cond1_baseline \
            --condition cond1_baseline \
```

```
    --hic-dir output/mcool \
    --n-jobs 4 --skip-existing --no-msd
```

Selected flags:

Flag	What it does
`--results-dir DIR`	Where to find merged simulation directories.
`--output-dir DIR`	Where to write outputs.
`--condition NAME`	Analyse one condition only.
`--n-jobs N`	Parallel workers for contact detection (default `analysis.n_jobs`).
`--skip-existing`	Skip dirs that already have `sim_contact_map.npy`.
`--no-msd`	Skip MSD computation (the Snakemake rule adds this unless `analysis.msd: True`).
`--hic-dir DIR`	Where to find `<compare_hic>.cool` for each condition (falls back to `output/mcool/`).
`--mcool TYPE=PATH`	Override the `.cool` for one Hi-C type (repeatable).
`--ctcf-bed TYPE=PATH`	Override the oriented CTCF BED for one ctcf_type (repeatable).

Per-replicate outputs (in `<sim_dir>/analysis/`):

File	What it is
`sim_contact_map.npy`	2D contact matrix from 3D conformations
`sim_ps_curve.npz`	Contact probability vs genomic distance
`sim_insulation.npy`	Insulation score along the locus
`apa_convergent.png`	Aggregate Peak Analysis at convergent CTCF pairs
`sim_vs_exp_map.png`	Side-by-side sim vs experimental Hi-C heatmap
`comparison_metrics.json`	Pearson + stratum-adjusted correlations with experimental Hi-C
`msd_<label>.npz`	Two-point MSD curves (if `--no-msd` not set)
`calibration.json`	Physical-unit mapping: nm/monomer, sec/frame

The calibration anchor is `analysis.calibration.default_anchor` (the `--calibrate-with` default). Experimental MSD CSVs come from `calibration.msd.expt_msd`, keyed by condition name or `ctcf_type` (`expt_msd_for`), plus any `--expt-msd`. With `calibration.msd.reference_condition`, every other condition loads that

condition's `*_pooled_calibration.json` (`reference_calibration_for`) instead of calibrating on its own, and the Snakefile makes its `analyzeCondition` wait for the reference's.

6b. Analysis modules (`analysis/`)

Module	Purpose
`contact_maps.py`	Contact matrix from conformations via `scipy.spatial.cKDTree`; also P(s), insulation, map-vs-map correlation
`ps_curve.py`	Power-law fitting of P(s)
`absolute_quant.py`	Aggregate Peak Analysis (APA) at convergent CTCF pairs
`ctcf_plotting.py`	Contact-map heatmaps with CTCF arrow strips
`experimental_compare.py`	Sim vs experimental Hi-C comparison (`.cool` and `.npy`)
`msd_two_point.py`	Mean-squared displacement of monomer pairs; sub-diffusive exponent fit
`polymer_dynamics.py`	Radius of gyration, loop fractions, dwell-time distributions
`calibration.py`	Map simulation units → physical units (nm/monomer, sec/frame)
`batch_analysis.py`	Older single-pass analysis over all result dirs (simpler than the orchestrator)

6c. `scripts/summarize_analysis.py`

Collects scalar metrics from all conditions into one CSV/XLSX table:

```
python scripts/summarize_analysis.py \
    --analysis-dir output/loci/chr1_fig1/analysis \
    --reference-condition cond1_baseline
```

6d. `scripts/check_results.py`

Lists every directory under a locus' `sims/` folder, its frame count, and whether analysis outputs are present. Quick inventory after a SLURM batch.

Legacy cluster scripts (`legacy/cluster/`)

The pre-Snakemake SLURM scripts were written for the lab's original cluster (mambaforge under `ric.tiget`, `cuda` / `workq` partitions) and the mESC/neuron study, so they have been moved to `legacy/cluster/` and are not maintained. **Use `sbatch ChromSimPipe.sh`**, which handles the full DAG on Longleaf.

`afterok` vs `afterany`

- `--dependency=afterok:<JID>`: fire only if upstream finished with exit 0. Use between pipeline stages where failure upstream makes the next stage meaningless.
- `--dependency=afterany:<JID>`: fire regardless of how upstream ended. Use for resume-chain links meant to recover from wall-time TIMEOUTs. `submit_analysis_resume_chain.sh` uses this pattern.

Conventions

- **Monomer = `resolution` bp (1 kb by default).** Monomer index `i` covers `REGION_START + i * RESOLUTION` to `REGION_START + (i+1) * RESOLUTION`.
- **Tiled chromosome.** Each locus is simulated on one chain of up to `tiling.chain_size` (70,000) monomers holding as many padded copies of it as fit (28 for the 1,960 kb chr1 locus). Contact maps are averaged back to the locus' own size for comparison, giving ~28× more contact statistics at no extra physics cost. `locus_tiling(name)` returns each locus' numbers.
- **Output directory names.** Shards: `output/loci/<locus>/sims/<params_name>_ctcf-<ctcf_type>_rep<N>_shard<S>/`. Merged: `output/loci/<locus>/sims/merged_<params_name>_ctcf-<ctcf_type>_rep<N>/`. The `_ctcf-<type>` part separates runs with the same cohesin params but different CTCF tracks.
- **RNG seed.** `seed_base + replicate * 10000 + shard * 1000` (`seed_base` = 42 in the config). Always derived from indices so reruns are reproducible.
- **Every run writes `params.json`** with the full parameter dict + CTCF type + replicate. If a result looks wrong, check `params.json` first.
- **Completion markers.** Snakemake tracks simulation, merge, and analysis steps through `.done` files in `output/done/`. Delete the relevant `.done` file if you need to re-run a step (for example after changing a parameter that only the scripts read).

Common pitfalls

CUDA architecture error. `openmm.OpenMMException: nvrtc: error: invalid value for --gpu-architecture (-arch)` means the environment's CUDA cannot compile for the allocated GPU. CUDA 13 dropped Pascal (GTX 10xx) and Volta (V100); `envs/cohesin_sim.yml` pins `cuda-version=12.4` for this reason. Check with `conda list -p <env> | grep cuda-version` and rebuild if it says 13.

`CUDA_ERROR_UNSUPPORTED_PTX_VERSION`. The environment's CUDA is newer than the GPU node's NVIDIA driver supports. `nvidia-smi` on a GPU node prints "CUDA Version", the most the driver supports; `cuda-version` in `envs/cohesin_sim.yml` must not exceed it (12.4 needs a driver of 550 or newer).

No Hi-C comparison in the analysis output. The analysis catches errors in each step, logs a warning and continues, so check `output/logs/analyzeCondition_*.err` for `WARNING`. A missing normalization column (`no 'KR' normalization`) is fixed with `analysis.hic_balance` (`auto`, or a column it lists).

Snakemake lock error after a failed run. Run `bash unlock.sh`, then resubmit.

`ModuleNotFoundError: configs` or `analysis`. Every script in `scripts/` inserts the repo root onto `sys.path`. Run from the repo root, or set `PYTHONPATH=/path/to/ChromSimPipe`.

`Samplesheet ... has no 'CTCF_Peaks' column.` `ChromSimSamplesheet.txt` must be tab-separated and include the headers `CTCF_Peaks` and `Hi-C`, plus every column named in `fileNamesFrom`.

`convertHic` fails. Check the sample's `Hi-C` path and format (`.hic`, `.mcool`, `.cool`) in the samplesheet, and `output/logs/convertHic_<sample>.err` (an `.mcool` without the config's `resolution` is named there).

Analysis fails with "No results found". Check that `output/loci/<locus>/sims/merged_*_rep*/` directories exist. If shards completed but the merged directory is missing, delete `output/done/mergeShards_<condition>_rep<N>.done` and resubmit.

No emails. Check `$CHROMSIM_EMAIL` is set in the shell you submit from (sbatch passes it on), and that `echo test | mail -s test $CHROMSIM_EMAIL` works on the cluster; otherwise set `notify: command:` to what does.

`Conda must be version 24.7.1 or later`. The cluster's conda is older than Snakemake 8.27 accepts (true of Longleaf's and Pioneer's modules). Install Miniforge once (`bash Miniforge3-Linux-x86_64.sh -b -p`

`$HOME/miniforge3`), delete the launcher's `env/` folder, and rerun; the shipped `env.sh` files activate it.

`Cannot load module ... because these module(s) are loaded.` Some clusters (Longleaf among them) refuse to load a `python` module and an `anaconda` module together. Load only the conda distribution in that site's `env.sh`; it brings its own Python.

Where the logs are. Each job's stdout / stderr: `output/logs/<rule>_<wildcards>.out|.err`. SLURM's own job logs: `output/logs_slurm/`. The launcher (main Snakemake process): `ChromSimPipe-<jobid>.out` in the repo root.